

Splunk: The Basics

Notes by Imon · September 2026 · github.com/imon333/SOC_Level_1

Task 1 – Introduction

After learning what a SIEM does in general, this room is the first hands-on look at an actual one. Splunk is one of the leading SIEM solutions: it collects, analyses and correlates network and machine logs in real time, which gives better visibility of what is happening in the network and speeds up detection.

What I wanted to get out of this room:

- The main components of Splunk
- How to find my way around the Splunk interface
- How log ingestion works in Splunk
- Ingesting real logs myself and searching through them

Prerequisite: the Introduction to SIEM room (chapter 02 in this repo).

Task 2 – Connect with the Lab

Setup only: start the lab machine (takes about 3–5 minutes) and open the Splunk instance in the browser. Everything in Tasks 4 and 5 was done on this instance.

Task 3 – Splunk Components

Splunk has three main components that work together. The easiest way for me to remember them is as a pipeline: **Forwarder** → **Indexer** → **Search Head**. Data is collected, then processed and stored, then searched.

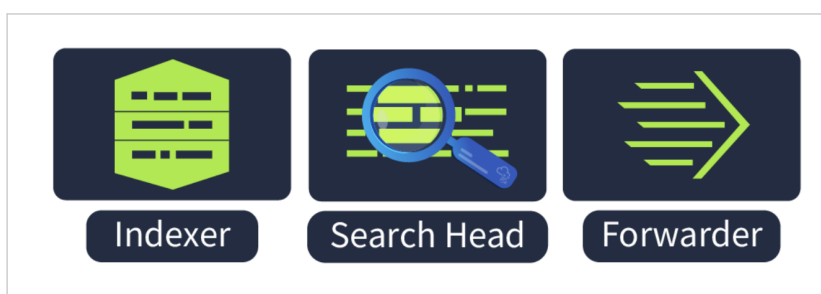


Figure 1: The three main Splunk components.

1. Forwarder

A lightweight agent installed on the endpoint we want to monitor. Its only job is to collect data and send it to the Splunk instance. It uses very few resources, so it doesn't slow the endpoint down. This is the same Agent/Forwarder ingestion method from the SIEM room. Typical data sources:

- Web servers generating web traffic logs
- Windows machines generating Windows Event Logs, PowerShell and Sysmon data
- Linux hosts generating host-centric logs
- Databases generating connection requests, responses and errors

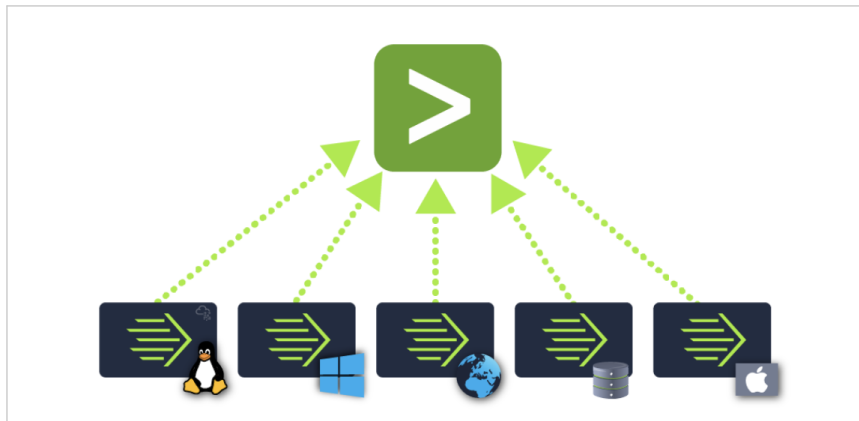


Figure 2: Forwarders on Linux, Windows, web, database and macOS hosts sending data to Splunk.

2. Indexer

The indexer does the heavy processing. It takes the data from the forwarders, parses and normalises it into **field-value pairs**, categorises it, and stores the results as **events**. After this step the data is easy to search and analyse. This is where the parsing and normalisation from the SIEM room actually happens.

3. Search Head

The search head is where I, as the analyst, work. It lives inside the **Search & Reporting** app. Searches are written in **SPL (Search Processing Language)**. When I run a search, the request goes to the indexer, and the matching events come back as field-value pairs.

```
index=main
```

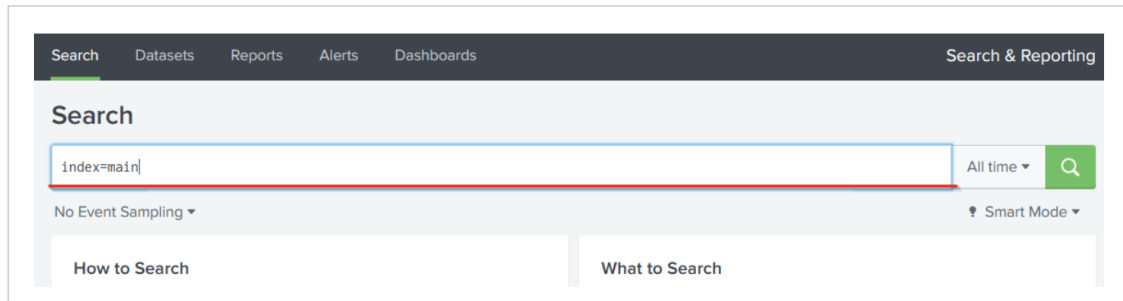


Figure 3: Searching the main index in Search & Reporting.

The search head can also turn results into tables and charts (pie, bar, column and more) under the Visualization tab. Each chart type even shows the SPL fragment it needs, for example `| stats count by comparison_category` for a column chart.

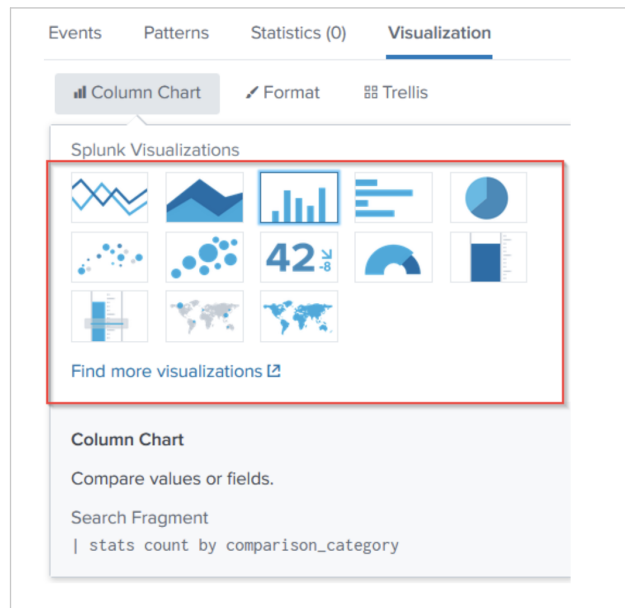


Figure 4: Visualization options in the search head.

Room questions

Which component is used to collect and send data over the Splunk instance?

Forwarder

Task 4 – Navigating Splunk

The home screen has four parts: the Splunk Bar at the top, the Apps Panel on the left, Explore Splunk in the middle, and the Home Dashboard below it.

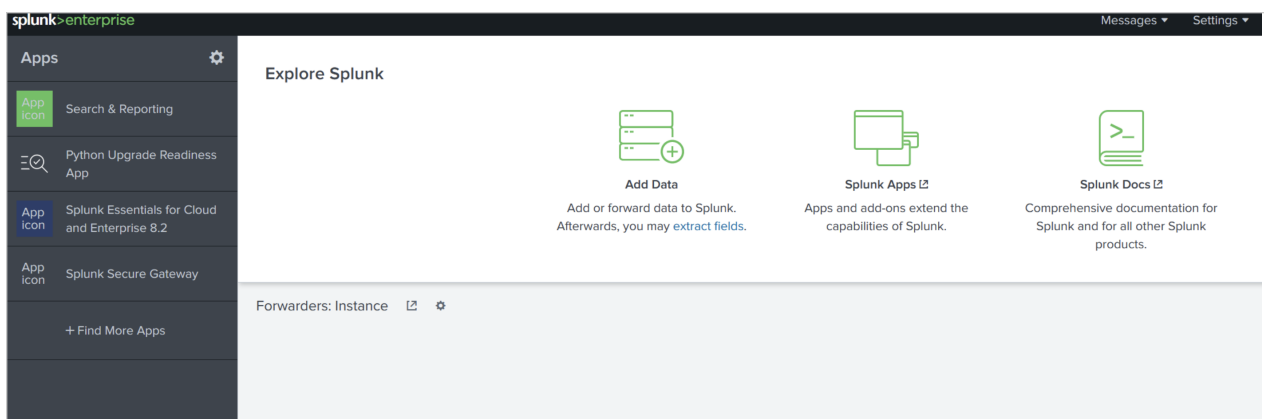


Figure 5: Splunk home screen with the Apps Panel and Explore Splunk.

Splunk Bar



Figure 6: The Splunk Bar.

Option	What it does
Messages	System-level notifications and messages
Settings	Configure the Splunk instance
Activity	Progress of search jobs and processes
Help	Tutorials and documentation
Find	Search across the app

The Splunk Bar can also be used to switch between installed apps, so the Apps Panel is not the only way.

Apps Panel

Shows the apps installed on the instance. The default app on every Splunk installation is **Search & Reporting**, which is where most of the analyst work happens.

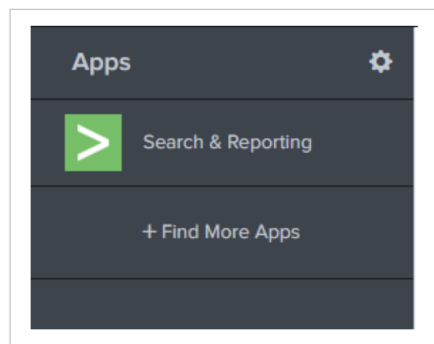


Figure 7: Apps Panel on a fresh install.

Explore Splunk

Quick links to **Add Data** (used in Task 5), **Splunk Apps** for add-ons, and **Splunk Docs**.

Home Dashboard

Empty by default. A dashboard can be picked from the dropdown or from the dashboards listing page. Dashboards I create myself show up separately under the *Yours* tab.

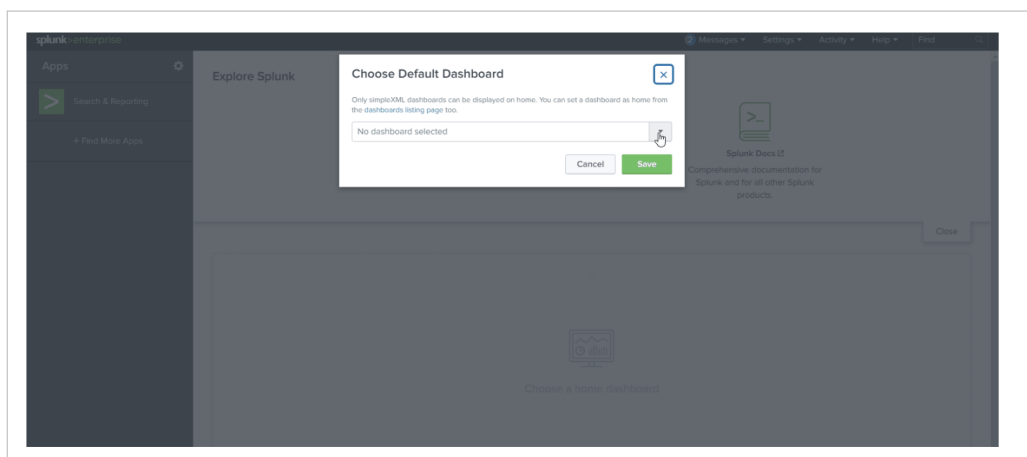


Figure 8: Choosing a default home dashboard.

Room questions

In the Add Data tab, which option is used to collect data from files and ports?

Monitor

Task 5 – Adding Data

Splunk can ingest basically any data. When data is added, Splunk processes it and breaks it into individual events. The Splunk docs group data sources into these categories:

Data source	Description
Files and directories	Most data that you might be interested in comes directly from files and directories.
Network events	The Splunk software can index remote data from any network port and SNMP events from remote devices.
IT Operations	Data from IT Ops, such as Nagios, NetApp, and Cisco.
Cloud services	Data from Cloud services, such as AWS and Kinesis.
Database services	Data from databases such as Oracle, MySQL, and Microsoft SQL Server.
Security services	Data from security services such as McAfee, Microsoft Active Directory, and Symantec Endpoint Protection.
Virtualization services	Data from virtualization services such as VMWare and XenApp.
Application servers	Data from application servers such as JMX & JMS, WebLogic, and WebSphere.
Windows sources	The Windows version of Splunk software accepts a wide range of Windows-specific inputs, including Windows Event Log, Windows Registry, WMI, Active Directory, and Performance monitoring.
Other sources	Other input sources are supported, such as FIFO queues and scripted inputs for getting data from APIs, and other remote data interfaces.

Figure 9: Data source categories from the Splunk documentation.

Clicking **Add Data** opens the page below. The top part has onboarding guides for common sources, the bottom has the three ingestion methods. **Upload** is for files from my own computer, **Monitor** watches files and ports on the Splunk instance itself, and **Forward** receives data from a forwarder.

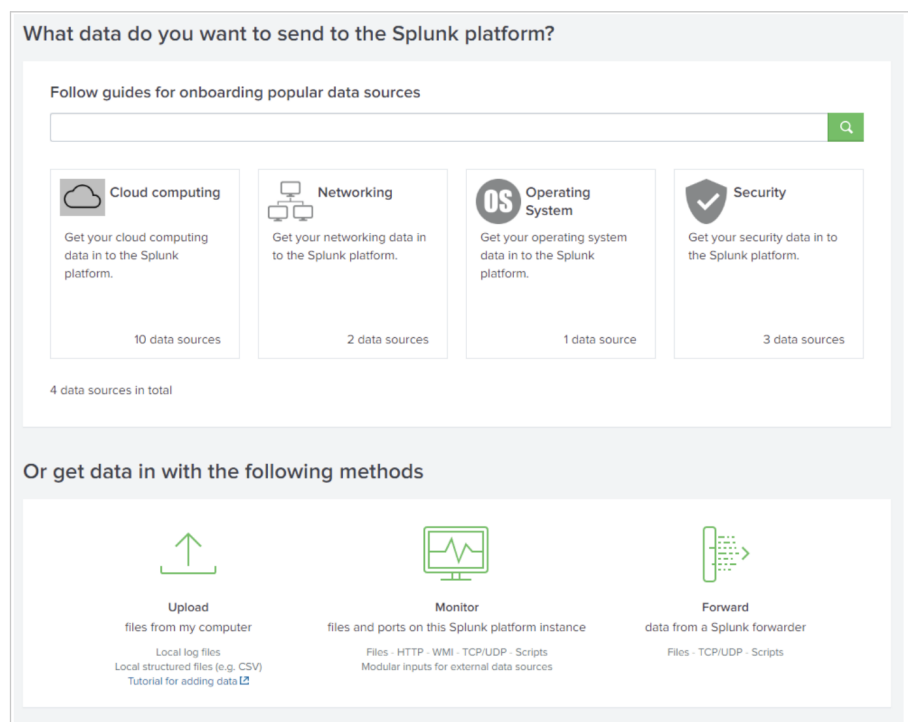


Figure 10: The Add Data page.

Practical: uploading VPN logs

The task file is a newline-delimited JSON file with VPN logs, so each line becomes one event. I used **Upload** and went through the five steps of the wizard:

Step	What I did
1. Select Source	Picked the downloaded file 03_logdata_VPN-logs-...json
2. Set Source Type	Kept the source type Splunk detected: _json. The preview already shows the parsed fields
3. Input Settings	Set the host value to VPN_Connection and created a new index for the logs
4. Review	Checked all settings
5. Done	File uploaded, then Start Searching

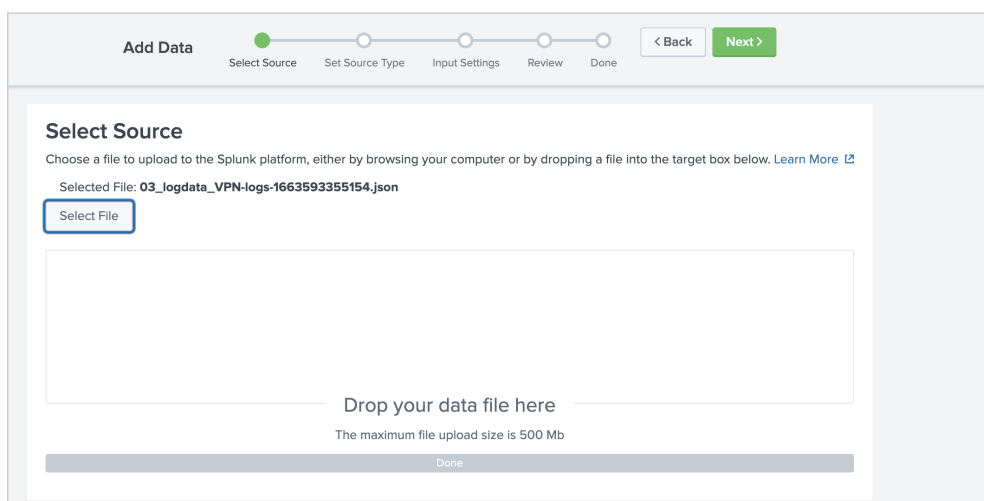


Figure 11: Step 1, selecting the log file.

The Set Source Type step is useful because it shows how Splunk will see the data *before* indexing. Here the JSON was already split into fields like action, Company, EventTime, port, protocol, Source_Country and Source_ip, so no extra configuration was needed.

Set Source Type

This page lets you see how the Splunk platform sees your data before indexing. If the events look correct and have the right timestamps, click "Next" to proceed. If not, use the options below to define proper event breaks and timestamps. If you cannot find an appropriate source type for your data, create a new one by clicking "Save As".

Source: 03_logdata_VPN-logs-1663593355154.json [View Event Summary](#)

Source type: _json [Save As](#)

	Table	Format	20 Per Page							
		_time	action	Company	EventTime	extracted_Index	port	protocol	Source_Country	Source_ip
1		01/01/2022 07:58:42.000	built	CyberT	2022-01-01T07:58:42	VPN_Logs	443	tcp	United States	167132.179.121
2		01/01/2022 17:26:59.000	teardown	CyberT	2022-01-01T17:26:59	VPN_Logs	443	tcp	United States	167132.179.121
3		01/01/2022 07:10:01.000	built	CyberT	2022-01-01T07:10:01	VPN_Logs	443	tcp	United States	151164.65.80
4		01/01/2022 17:45:39.000	teardown	CyberT	2022-01-01T17:45:39	VPN_Logs	443	tcp	United States	151164.65.80
5		01/01/2022 08:52:55.000	built	CyberT	2022-01-01T08:52:55	VPN_Logs	443	tcp	United States	10714.105.104
6		01/01/2022 16:07:19.000	teardown	CyberT	2022-01-01T16:07:19	VPN_Logs	443	tcp	United States	10714.105.104
7		01/01/2022 07:58:44.000	built	CyberT	2022-01-01T07:58:44	VPN_Logs	443	tcp	United States	151164.57.163
8		01/01/2022 18:21:24.000	teardown	CyberT	2022-01-01T18:21:24	VPN_Logs	443	tcp	United States	151164.57.163
9		01/01/2022	built	CyberT	2022-01-	VPN_Logs	443	tcp	United States	107.5.232.195

Figure 12: Step 2, preview of the parsed events with source type _json.

In Input Settings, the **host** value should be the name of the machine the events come from. The **index** is where the events are stored. Splunk also suggests using a separate "sandbox" index when testing, so production indexes are not affected.

Input Settings

Optionally set additional input parameters for this data input as follows:

Host

When the Splunk platform indexes data, each event receives a "host" value. The host value should be the name of the machine from which the event originates. The type of input you choose determines the available configuration options. [Learn More](#)

Host field value:

Index

The Splunk platform stores incoming data as events in the selected index. Consider using a "sandbox" index as a destination if you have problems determining a source type for your data. A sandbox index lets you troubleshoot your configuration without impacting production indexes. You can always change this setting later. [Learn More](#)

Index: [Create a new index](#)

FAQ

- > How do indexes work?
- > How do I know when to create or use multiple indexes?

Figure 13: Step 3, host set to VPN_Connection and the new index selected.

Figure 14: Creating a new index. Only the name is required, the rest can stay default.

Side note: the room asks for an index called `VPN_Logs`. I named mine `VPN_Log`, which is why the searches below use `index="VPN_Log"`. The results are the same.

Figure 15: Step 5, upload complete.

Searching the data

With the time picker set to **All time**, searching the whole index returns every event:

```
index="VPN_Log"
```

2,862 events. The left sidebar lists the extracted fields, and each event is shown as JSON with fields like `UserName`, `Source_ip`, `Source_Country` and `action` (built/teardown for VPN session start and end).

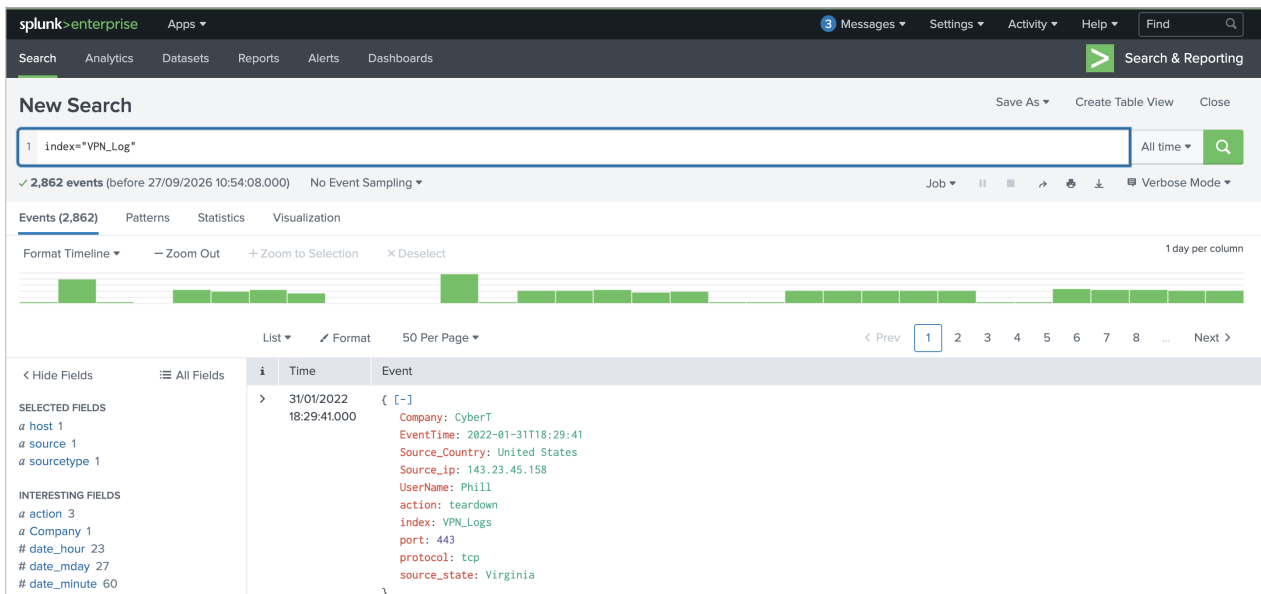


Figure 16: All events in the index, 2,862 in total.

Adding a keyword filters the results. Searching for a username:

index="VPN_Log" Maleena

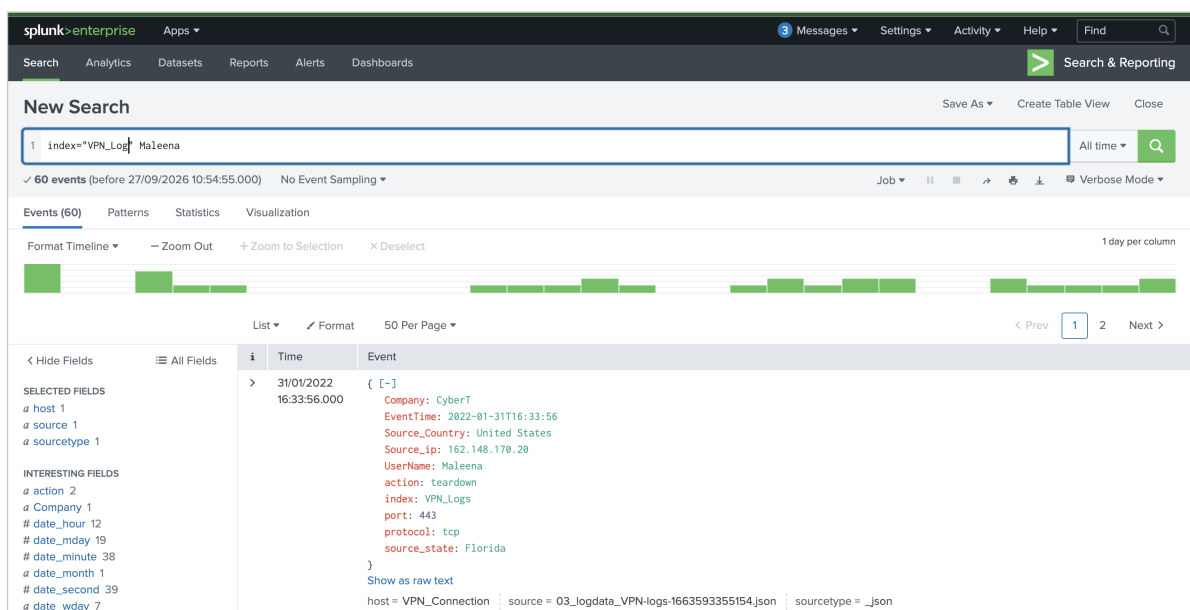


Figure 17: 60 events for the user Maleena.

The same works for an IP address. For 107.14.182.38 there are 26 events, and clicking the UserName field in the sidebar shows that all of them belong to one user, **Smith**. The field popup is a quick way to see the distribution of values without writing a stats query.

index="VPN_Log" "107.14.182.38"

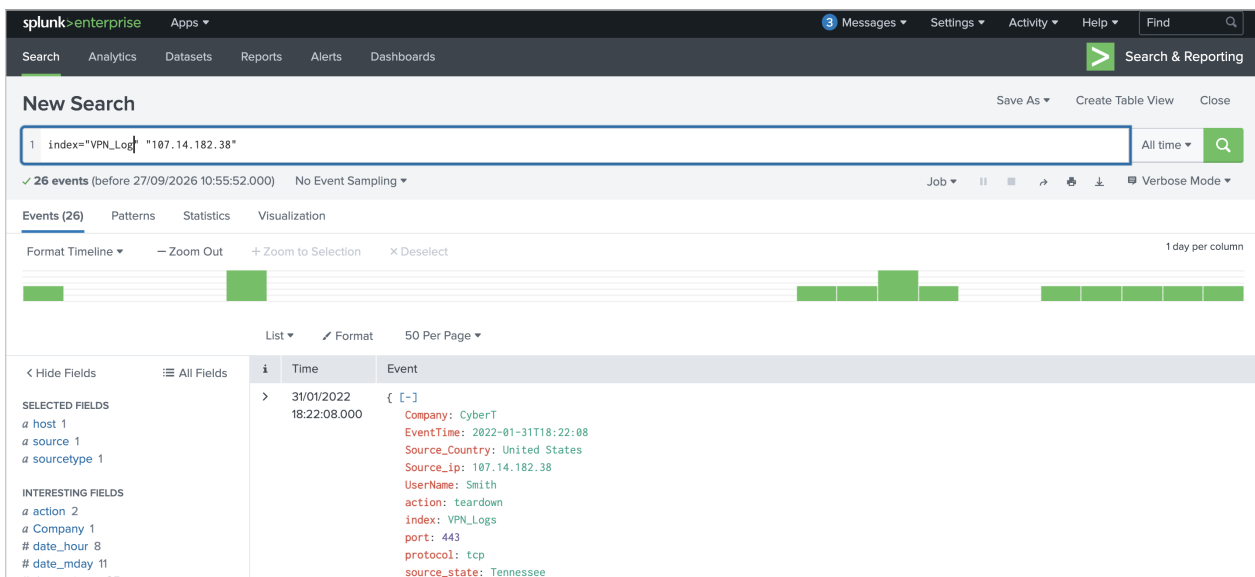


Figure 18: Events for IP 107.14.182.38.

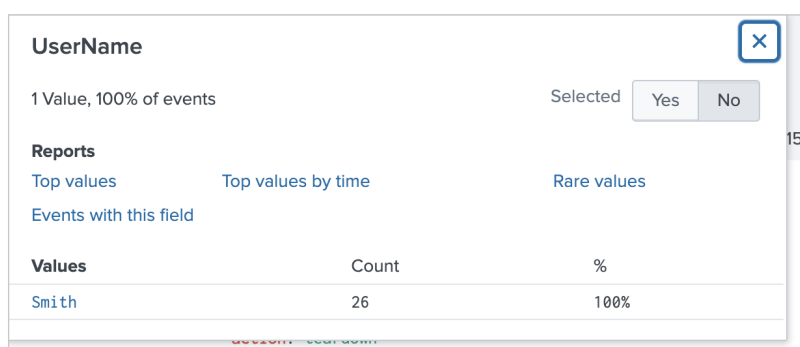


Figure 19: Username field summary: Smith, 26 events (100%).

For 107.3.206.58 the search returns **14 events**. Interesting detail: the username here is *Will Smith*, a different user from the "Smith" above, even though the names look similar. Worth keeping in mind: always check the exact field value and don't assume by name.

```
index="VPN_Log" "107.3.206.58"
```

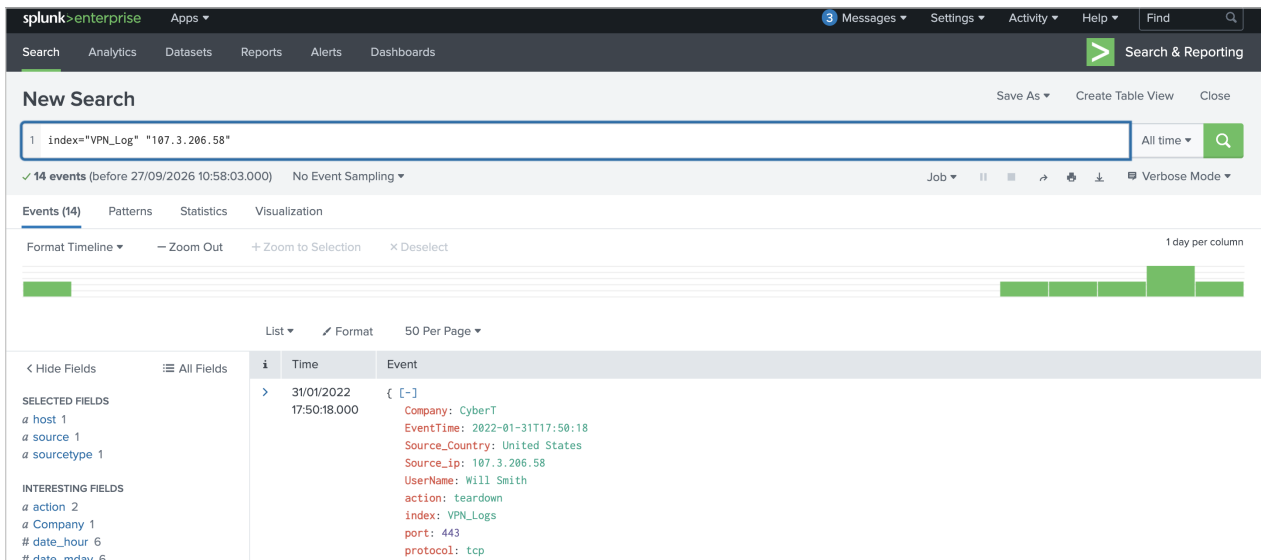


Figure 20: 14 events for IP 107.3.206.58.

Keyword search vs. field search

The searches above are plain keyword searches: Splunk looks for the text anywhere in the event. That works here, but it could also match the same string in a different field. Searching on the field itself is more precise. The room also gave these field-based versions (spath makes sure the JSON fields are extracted):

```
index=VPN_Logs | stats count
index=VPN_Logs | spath | search UserName="Maleena" | stats count
index=VPN_Logs | spath | search Source_ip="107.14.182.38" | stats values(UserName) as UserName count
index=VPN_Logs | spath | search Source_Country!="France" | stats count
index=VPN_Logs | spath | search Source_ip="107.3.206.58" | stats count
```

The France question gives **2,814**, so $2,862 - 2,814 = 48$ events came from France.

Side note: `Source_Country!="France"` only returns events that *have* a `Source_Country` field with a different value. `NOT Source_Country="France"` would also include events where the field is missing. In this dataset every event has the field, so both give the same result, but on real data the difference matters.

Room questions

How many events are present in the log file?	2862
How many log events are captured by the user Maleena?	60
What is the username associated with IP 107.14.182.38?	Smith
Number of events from all countries except France?	2814
How many VPN events were associated with IP 107.3.206.58?	14

Task 6 – Conclusion

This room made the SIEM concepts concrete for me. The pipeline Forwarder → Indexer → Search Head maps directly to collect, normalise and search from the previous room. Uploading the VPN logs showed how much work the indexer does automatically: the JSON was parsed into fields before I even finished the wizard. Searching with keywords is quick, but field-based SPL with `stats` is more precise, and that is what I want to get better at next.

Rooms suggested for going deeper: Splunk: Exploring SPL, Incident Handling with Splunk, Investigating with Splunk, Benign (challenge), PoshEclipse (challenge).